

UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS

Facultad de Ingeniería

METODOLOGÍAS ÁGILES EN EL DESARROLLO DE SOFTWARE

Estudiantes / Autores:

Sebastián Avendaño Rodríguez
Cód. 20232020101

Juan Sebastián Rodríguez Serrano
Cód. 20232020038

Manuel Arturo Rueda Guiza
Cód. 20232020052

Asignatura:

Fundamentos de Ingeniería de
Software

Programa Académico:

Ingeniería de Sistemas

Bogotá D.C., Colombia

14 de septiembre de 2026

Índice

1. Principios y Fundamentos de Agile	2
2. Extreme Programming (XP)	2
3. Scrum	3
4. Kanban	3
5. Crystal	4
6. Comparación entre XP, Scrum, Kanban y Crystal	5
7. Aplicación Práctica: Caso de Estudio Comparado	5
7.1. Selección justificada de la solución técnica	6
8. Conclusiones y Guía Rápida de Decisión	6

1. Principios y Fundamentos de Agile

El desarrollo ágil surge como respuesta a las limitaciones del modelo en cascada tradicional, en el cual la rigidez documental y la postergación de la validación generaban altos índices de fracaso ante requisitos cambiantes (Boehm, 1981). Frente a esto, el *Manifiesto Ágil* (Beck et al., 2001) formalizó un cambio de paradigma centrado en cuatro valores axiológicos:

1. **Individuos e interacciones** sobre procesos y herramientas.
2. **Software funcionando** sobre documentación extensiva.
3. **Colaboración con el cliente** sobre negociación contractual.
4. **Respuesta ante el cambio** sobre seguir un plan preestablecido.

Estos valores se operacionalizan en doce principios agrupados en cuatro dimensiones: entrega temprana y continua de valor observable; adaptabilidad como ventaja competitiva; equipos motivados, autoorganizados y con comunicación directa cara a cara; y búsqueda de excelencia técnica, diseño simple y ritmo de desarrollo sostenible. En la industria contemporánea, estos fundamentos sustentan prácticas como la integración y despliegue continuos (CI/CD), la cultura DevOps y las arquitecturas desacopladas orientadas al usuario.

2. Extreme Programming (XP)

Formulada por Kent Beck (1999), Extreme Programming es una metodología ágil orientada a la excelencia técnica del código fuente. Se fundamenta en cinco valores (comunicación, simplicidad, retroalimentación, coraje y respeto) y articula doce prácticas de ingeniería sinérgicas:

- **Prácticas técnicas centrales:** Desarrollo Guiado por Pruebas (TDD, ciclo *Red-Green-Refactor*), Programación en Parejas (*Pair Programming*), Refactorización continua, Integración Continua (CI), Diseño Simple (principio *YAGNI*) y Propiedad Colectiva del Código.
- **Prácticas de gestión y equipo:** El Juego de la Planificación, Entregas Pequeñas, Estándares de Codificación, Ritmo Sostenible (semanas de 40 horas sin sobretiempo crónico) y la presencia obligatoria de un Cliente In Situ (*On-site Customer*).
- **Roles:** Desarrollador (*Programmer*), Cliente In Situ, Rastreador (*Tracker*) y Entrenador (*Coach*).
- **Balance:** Reduce drásticamente la tasa de defectos y la deuda técnica, pero presenta una fuerte dependencia de la disponibilidad del cliente y suele provocar fatiga si la programación

en parejas se ejecuta sin rotación adecuada.

- **Uso óptimo:** Proyectos con alta volatilidad funcional, complejidad algorítmica y equipos reducidos (≤ 12 ingenieros) con alta madurez técnica.

3. Scrum

Scrum es un marco de trabajo (*framework*) empírico e iterativo estructurado por Ken Schwaber y Jeff Sutherland (2020) para resolver problemas complejos adaptativos, apoyado en tres pilares: **transparencia**, **inspección** y **adaptación**. Prescribe tres responsabilidades, cinco eventos y tres artefactos formalmente acoplados a compromisos:

- **Responsabilidades (Scrum Team):**
 - *Product Owner*: Maximiza el valor del producto y prioriza de forma exclusiva el Product Backlog.
 - *Scrum Master*: Líder al servicio del equipo; vela por el cumplimiento del marco y remueve impedimentos.
 - *Developers*: Profesionales autónomos encargados de crear un incremento utilizable en cada ciclo.
- **Eventos (bloques de tiempo o *timeboxes*):** El Sprint (1 a 4 semanas), Sprint Planning (define el objetivo y alcance del ciclo), Daily Scrum (sincronización diaria de 15 minutos), Sprint Review (demostración del incremento ante interesados) y Sprint Retrospective (mejora continua del proceso interno).
- **Artefactos y Compromisos:** El *Product Backlog* (compromiso: Objetivo del Producto), el *Sprint Backlog* (compromiso: Objetivo del Sprint) y el *Increment* (compromiso: Definición de Terminado / *Definition of Done*).
- **Balance:** Proporciona alta visibilidad organizacional y alineación de negocio; no obstante, carece de prescripciones de ingeniería de software, lo que arriesga la acumulación de deuda técnica si el equipo no adopta prácticas de desarrollo rigurosas.
- **Uso óptimo:** Desarrollo de productos comerciales, sistemas SaaS y proyectos donde los requisitos pueden empaquetarse en metas periódicas predecibles.

4. Kanban

Adaptado al software por David J. Anderson (2010) a partir del Sistema de Producción Toyota (Ohno, 1988), Kanban es un método no prescriptivo enfocado en la optimización del flujo

continuo de trabajo mediante sistemas de extracción (*pull*):

- **Principios operacionales:** Comenzar con lo que se hace ahora (respeto a roles existentes), promover la mejora continua incremental (*Kaizen*) y gestionar el trabajo mediante visualización explícita en un Tablero Kanban.
- **Límites de Trabajo en Progreso (WIP):** Restringen deliberadamente la cantidad de tareas activas por columna para evitar sobrecargas cognitivas. Se sustenta formalmente en la **Ley de Little**:

$$W = \frac{L}{\lambda} \quad (1)$$

donde W es el tiempo de entrega (*Lead Time*), L es el inventario en proceso (WIP) y λ es el rendimiento (*Throughput*). De la ecuación se deduce que reducir el WIP (L) es la vía matemáticamente directa para reducir el tiempo de respuesta del sistema (W).

- **Métricas clave:** *Lead Time*, *Cycle Time* y el Diagrama de Flujo Acumulado (CFD) para identificar visualmente atascos y colas de espera.
- **Balance:** Máxima flexibilidad ante interrupciones imprevistas y baja fricción organizacional; sin embargo, no incluye ceremonias de sincronización obligatorias ni lineamientos de diseño técnico.
- **Uso óptimo:** Soporte de incidentes, mantenimiento de software legado y operaciones continuas de infraestructura / DevOps.

5. Crystal

Propuesta por Alistair Cockburn (2004), Crystal es una familia de metodologías adaptables centradas en el factor humano, la comunicación directa y la tolerancia metodológica.

- **Matriz de Cockburn:** Clasifica los proyectos en función de dos variables: el tamaño del equipo (representado por gemas y colores progresivamente más oscuros: Clear, Yellow, Orange, Red) y la criticidad del impacto ante defectos:
 - **C** (Comodidad / Confort), **D** (Dinero Discrecional), **E** (Dinero Esencial) y **L** (Vida Humana / *Life*).
- **Variantes principales:** Desde *Crystal Clear* (1 a 8 personas, criticidades C–E, apoyada en comunicación osmótica y documentación mínima) hasta variantes *Orange/Red* o *Maroon* (decenas de ingenieros, criticidad L), donde se exigen especificaciones formales de interfaz, auditorías de trazabilidad y verificaciones independientes.
- **Propiedades centrales:** Entrega frecuente, mejora reflexiva, comunicación osmótica y seguridad psicológica en el equipo.

- **Balance:** Evita la sobrecarga burocrática al modular el peso del proceso según el riesgo real; en contrapartida, cuenta con baja difusión comunitaria y exige un criterio maduro para calibrar el método sin caer en la desorganización.
- **Uso óptimo:** Organizaciones con portafolios heterogéneos que abarcan desde prototipos internos hasta software con criticidad financiera o de seguridad.

6. Comparación entre XP, Scrum, Kanban y Crystal

Criterio	XP	Scrum	Kanban	Crystal (Clear/Orange)
Objetivo	Calidad de código y adaptabilidad técnica.	Valor de producto y gestión empírica.	Optimización del flujo continuo y lead time.	Adaptar el proceso a tamaño y criticidad.
Estructura	Alta en ingeniería de software.	Media (roles, eventos y artefactos de gestión).	Mínima; superpuesta al proceso actual.	Modular según la matriz de Cockburn.
Roles	Cliente, Programador, Tracker, Coach.	Product Owner, Scrum Master, Developers.	No prescribe nuevos roles formales.	Flexibles en Clear; formales en Orange/Red.
Gestión	Iterativa por Historias de Usuario.	Iterativa en timeboxes fijos (<i>Sprints</i>).	Flujo continuo tipo <i>pull</i> con límites WIP.	Iterativa basada en entregas frecuentes.
Iteración	1 a 2 semanas.	1 a 4 semanas.	Sin iteraciones obligatorias (continuo).	Semanas a meses (según riesgo).
Flexibilidad	Inmediata entre iteraciones.	Sin cambios que alteren el Sprint Goal.	Máxima; reordenamiento continuo de la cola.	Alta; ajustada en talleres reflexivos.
Tamaño eq.	2 a 12 desarrolladores.	3 a 9 devs (Scrum Team ≤ 10).	Cualquier tamaño organizacional.	Clear (1–8), Yellow (9–20), Orange (21–40).
Ventaja	Mínimos defectos; código limpio con TDD.	Foco en valor y visibilidad del progreso.	Fluidez operativa y reducción de cuellos.	Enfoque humano; evita burocracia inútil.
Desventaja	Desgaste en parejas; dependencia cliente.	Riesgo de deuda técnica sin ingeniería.	Riesgo de relajación sin autodisciplina.	Poca adopción actual en el mercado.
Proyecto	Lógica compleja o algoritmos críticos.	Nuevos productos SaaS y sistemas web.	Mantenimiento, soporte IT y DevOps.	Cartera mixta con criticidades diversas.

Tabla 1: Matriz comparativa sintetizada entre los enfoques evaluados.

7. Aplicación Práctica: Caso de Estudio Comparado

Se analiza el desarrollo de una **Plataforma Hospitalaria de Telemedicina y Recetas Digitales**, compuesta por tres subsistemas: el motor clínico de recetas (alta criticidad por interacción de fármacos), el portal web/móvil para pacientes (alta volatilidad de diseño UI/UX) y el módulo de soporte y operaciones continuas.

- **Extreme Programming (XP):** Idóneo para el *Motor de Recetas*. Se implementa TDD para verificar algorítmicamente dosificaciones e incompatibilidades farmacológicas antes de codificar, respaldado por programación en parejas y un médico especialista como Cliente In Situ.
- **Scrum:** Aplicado a la gobernanza general y al *Portal del Paciente*. Sprints de 2 semanas permiten que el Product Owner priorice funcionalidades de interfaz y videollamada, validando incrementos funcionales con usuarios reales en cada Sprint Review.
- **Kanban:** Asignado al *Módulo de Soporte y Operaciones*. Gestiona incidentes de infraestructura y sincronización con farmacias mediante un flujo continuo con límites de WIP y control de SLA, sin la rigidez de planificaciones fijas.
- **Crystal:** Al evaluar el riesgo de daño vital (Criticidad **L**), la matriz prescribe implementar *Crystal Yellow/Orange (L)*, exigiendo contratos de interfaz formales y auditorías de código independientes para el módulo farmacológico.

7.1. Selección justificada de la solución técnica

La solución técnica superior es un **modelo híbrido**: gobernar el producto integralmente bajo **Scrum** para asegurar la alineación con los interesados; aplicar la disciplina técnica de **XP** (TDD y Pair Programming) en el núcleo farmacológico crítico para mitigar riesgos vitales; y operar el mantenimiento e infraestructura bajo **Kanban** para absorber incidentes reactivos en tiempo real.

8. Conclusiones y Guía Rápida de Decisión

De acuerdo con el postulado de Frederick Brooks (1987) en *No Silver Bullet*, **no existe una metodología universalmente superior**. Su efectividad depende de la concordancia entre el contexto del problema y las características estructurales del método:

- **Seleccionar Extreme Programming (XP):** En equipos pequeños de alta disciplina técnica donde la tasa de defectos deba tender a cero y los requerimientos funcionales o algorítmicos sean altamente volátiles.
- **Seleccionar Scrum:** En desarrollo de productos comerciales y plataformas evolutivas que demanden coordinación periódica entre negocio y desarrollo bajo metas fijas de 1 a 4 semanas.
- **Seleccionar Kanban:** En entornos de demanda impredecible, soporte técnico, mesas de ayuda y operaciones DevOps donde los lotes por Sprint generen fricción operativa.

- **Seleccionar Crystal:** En organizaciones con proyectos de perfiles dispares que requieran calibrar conscientemente el peso del proceso según el número de personas y la gravedad del fallo (desde comodidad hasta riesgo de vidas).

Referencias

- [1] Anderson, D. J. (2010). *Kanban: Successful evolutionary change for your technology business*. Blue Hole Press.
- [2] Beck, K. (1999). *Extreme Programming Explained: Embrace change*. Addison-Wesley.
- [3] Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Highsmith, J., Hunt, A., Jeffries, R., Schwaber, K., Sutherland, J., & Thomas, D. (2001). *Manifesto for Agile Software Development*. Agile Alliance. <https://agilemanifesto.org/>
- [4] Boehm, B. W. (1981). *Software engineering economics*. Prentice-Hall.
- [5] Brooks, F. P. (1987). No silver bullet: Essence and accidents of software engineering. *IEEE Computer*, 20(4), 10–19. <https://doi.org/10.1109/MC.1987.1663532>
- [6] Cockburn, A. (2004). *Crystal Clear: A human-powered methodology for small teams*. Addison-Wesley Professional.
- [7] Ohno, T. (1988). *Toyota production system: Beyond large-scale production*. Productivity Press.
- [8] Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide: The definitive guide to Scrum: The rules of the game*. Scrum.org. <https://scrumguides.org/>